

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D046.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 046 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

完整流水线:数据→分析→图

// End-to-End Pipeline

KEY CONCEPTS · 三大要点

- ▶ 模块化
- ▶ 可复用
- ▶ 脚本化

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面我们学了一堆零散的本事:拉数据、清洗、算指标、画图、存数据库。但你大概也发现,每次研究都是把这些步骤复制粘贴一遍,改来改去乱成一团。这一节我们把它串成一条完整的流水线,像工厂的传送带,原料(行情)从一头进去,经过拉取、清洗、分析、画图一道道工序,成品(分析图)从另一头出来。我们把每道工序做成1块独立的『积木函数』,再用一个总开关串起来;以后想换只股票、换个区间,只改一处配置,整条线自动重跑。我们用格力电器和海尔智家两只票,演示同一条流水线怎么一键处理多只票。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解什么是流水线和模块化:把一大坨代码拆成 fetch/clean/analyze/plot 4 块独立积木
- 02 会把每道工序写成一个职责单一的函数,输入输出清清楚楚
- 03 会用一个总函数把四道工序串起来,一键处理一只票
- 04 理解可复用:把要改的参数集中到一个 CONFIG,换票换区间只动一处,整条线重跑
- 05 知道进阶上专门工具(命令行 click、配置 yaml、日志 loguru、定时 cron)各管什么

02 · CONTEXT · 历史与背景

小赵的研究代码越堆越乱,改一个地方崩三个地方

// 不读历史的策略走不远

小赵刚学完拉数据、画图,兴致勃勃写了个分析脚本。一开始还好,可慢慢地,他想多看几只票,就把代码复制一份改个代码;想换个时间段,又翻遍整个文件找哪里写了日期;想多算一个指标,结果一改,画图那段莫名其妙报错了。几百行代码搅成一锅粥,改一个地方崩三个地方,他自己都看不懂了。一位做工程的朋友看了直摇头:你这是把所有零件焊死在一起了。正确的做法是像搭积木,拉数据是1块、清洗是1块、分析是1块、画图是1块,每块管好自己的事,块和块之间用清楚的接口连接。要改哪一步,只动那1块,别的都不受影响。小赵照着把代码拆成四个函数、参数集中到一处,代码立刻清爽了,换票换区间也不再牵一发动全身。这就是模块化的威力:把复杂拆成简单,把混乱变成有序。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

流水线与模块化:把大问题拆成几块小积木

做一次完整的量化分析,要经过好几步:拉数据、清洗、分析、画图。新手容易把这些全写成一大坨,结果改哪都怕崩。流水线的思路是把它拆开:每一步做成1块独立的『积木』(在代码里就是一个函数),每块只管好自己那一摊,输入什么、输出什么清清楚楚。就像工厂的传送带,一道工序接一道工序。拆开之后,哪一步出问题就修哪1块,想升级哪一步就换哪1块,别的纹丝不动。这是写出能长期维护的代码的第一课。

CONCEPT_02

fetch 与 clean:第一、二道工序,拿到数据并洗干净

第1块积木 fetch 负责拉数据:你给它一个股票代码和时间段,它就去把原始行情取回来,别的不管。第2块积木 clean 负责清洗:刚拉回来的数据往往是字符串、没排序、可能有缺失, clean 把它转成数字、按日期排好、去掉缺失,交出一张干净规整的表。把『拿数据』和『洗数据』分成两块,好处是:以后换个数据源,只改 fetch;清洗规则要调整,只改 clean,互不打扰。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

analyze 与 plot:第三、四道工序,算出结论并画出来

第3 块积木 analyze 负责分析:接过干净的数据,算出你关心的东西,净值、均线、回撤,再顺手汇总几个关键指标(区间收益、最大回撤)。第4 块积木 plot 负责画图:把分析结果画成一张专业的图。注意它俩的分工,analyze 只算不画、plot 只画不算。这样你想换种算法,只动 analyze;想换种画法,只动 plot。职责单一,是好积木的标志。

CONCEPT_04

串起来 + 可复用:一个总开关,换票只改一处

4 块积木有了,用一个总函数 run_one 按顺序把它们串起来:fetch 拿数据 → clean 洗干净 → analyze 算结论 → plot 画图。这就是处理一只票的完整流水线。更妙的是可复用:把所有要改的参数(股票、起止日期、均线窗口)集中放进一个叫 CONFIG 的配置里,主程序对 CONFIG 里的每只票循环跑一遍同一条流水线。想多分析几只票、换个区间?只改 CONFIG 这一处,整条线自动重跑,一行重复代码都不用写。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

进阶:命令行、配置文件、日志、定时各管什么

等你的流水线要给别人用、要每天自动跑,有几个专门的工具值得认识(知道名字即可,这节先用最朴素的方式实现它们的功能)。click 帮你把脚本变成像专业软件那样的命令行,敲一行命令带参数就能跑;yaml 是一种把配置单独写进一个文件的格式,改参数不用碰代码;loguru 是个好用的日志库,让程序边跑边清楚地告诉你它在干嘛一步(我们这节先用 print 代替);cron 是操作系统的定时器,能让你的流水线每天收盘后自动跑一遍。先理解思想,工具是后话。

04 · PYTHON LAB · 真实可跑代码

完整流水线 — fetch / clean / analyze / plot 4 块积木 + CONFIG 配置 + run_one 串联(换票只改一处)

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_046_pipeline.py — 把零散步骤串成一条可复用的流水线:fetch→clean→analyze→plot
# 像工厂传送带:原料(行情)进去,一道道工序加工,成品(分析图)出来;换只股票整条线重跑
# 数据:baostock 日线(免费、国内零翻墙)。真名上屏:fetch / clean / analyze / plot /
CONFIG
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False

# ==== 配置集中放一处:想改什么(换票、换区间、换均线)只动这里 ====
CONFIG = {
    'stocks': {'格力电器': 'sz.000651', '海尔智家': 'sh.600690'},
    'start': '2022-01-01',
    'end': '2024-12-31',
    'ma_windows': [20, 60],
}

def log(msg):
    print(f'[流水线] {msg}')

# ---- 第一道工序:fetch 拉数据(输入代码,输出原始行情表)----
def fetch(code, start, end):
    log(f'fetch:拉取 {code} 的日线 ...')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/4)

完整流水线 — fetch / clean / analyze / plot 4 块积木 + CONFIG 配置 + run_one 串联(换票只改一处)

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
lg = bs.login()
if lg.error_code != '0':
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
rs = bs.query_history_k_data_plus(code, 'date,close,volume', start_date=start,
end_date=end, frequency='d', adjustflag='2')
rows = []
while rs.error_code == '0' and rs.next():
    rows.append(rs.get_row_data())
bs.logout()
return pd.DataFrame(rows, columns=['date', 'close', 'volume'])

# ---- 第二道工序:clean 清洗(转类型 + 日期索引 + 去缺失 + 排序)----
def clean(raw):
    log('clean:转数字 + 日期索引 + 去缺失 + 排序')
    df = raw.copy()
    df['date'] = pd.to_datetime(df['date'])
    df['close'] = pd.to_numeric(df['close'])
    df['volume'] = pd.to_numeric(df['volume'])
    return df.dropna().set_index('date').sort_index()

# ---- 第三道工序:analyze 分析(净值 + 均线 + 回撤,顺手算几个指标)----
def analyze(df, ma_windows):
    log('analyze:算净值 / 均线 / 回撤')
    out = df.copy()
    out['nv'] = out['close'] / out['close'].iloc[0]
    for w in ma_windows:
        out[f'ma{w}'] = out['close'].rolling(w).mean()
    out['dd'] = out['nv'] / out['nv'].expanding().max() - 1
    stats = {
        '区间收益%': round((out['nv'].iloc[-1] - 1) * 100, 1),
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/4)

完整流水线 — fetch / clean / analyze / plot 4 块积木 + CONFIG 配置 + run_one 串联(换票只改一处)

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
'最大回撤%': round(out['dd'].min() * 100, 1),
'交易日数': len(out),
}
return out, stats

# ---- 第四道工序:plot 出图(主图价格+均线, 副图回撤) ----
def plot(name, out):
    log(f'plot:画 {name} 的分析图')
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 7), sharex=True,
    gridspec_kw={'height_ratios': [3, 1]})
    ax1.plot(out.index, out['close'], label='收盘', lw=1, color='#1f3b6e')
    for c in [c for c in out.columns if c.startswith('ma')]:
        ax1.plot(out.index, out[c], label=c, lw=1.3)
    ax1.set_title(f'{name} 价格 + 均线 / 回撤(整条流水线一键产出)')
    ax1.set_ylabel('价格(元)'); ax1.legend(); ax1.grid(alpha=0.3)
    ax2.fill_between(out.index, out['dd'] * 100, 0, color='salmon', alpha=0.8)
    ax2.set_ylabel('回撤 %'); ax2.grid(alpha=0.3)
    plt.tight_layout(); plt.savefig(f'{name}.png', dpi=120)

# ---- 把四道工序串起来 = 处理一只票的完整流水线 ----
def run_one(name, code, cfg):
    log(f'=== 开始处理 {name} ===')
    raw = fetch(code, cfg['start'], cfg['end'])
    df = clean(raw)
    out, stats = analyze(df, cfg['ma_windows'])
    plot(name, out)
    log(f'{name} 完成:{stats}')
    return stats

# ==== 主程序:对配置里的每只票跑同一条流水线(可复用) ====
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/4)

完整流水线 — fetch / clean / analyze / plot 4 块积木 + CONFIG 配置 + run_one 串联(换票只改一处)

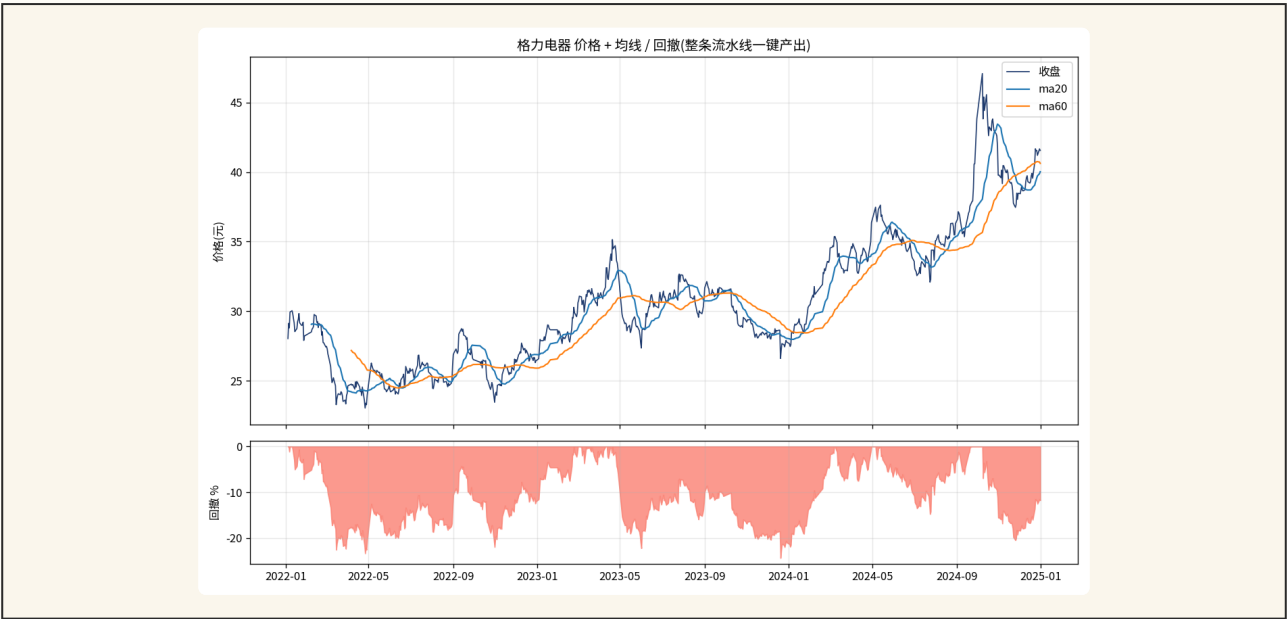
// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('==== 完整流水线:数据→清洗→分析→画图,一键跑完 ====')
results = {}
for nm, cd in CONFIG['stocks'].items():
    results[nm] = run_one(nm, cd, CONFIG)

print('\n==== 汇总:两只票跑完同一条流水线的结果对比 ====')
summary = pd.DataFrame(results).T
print(summary.to_string())
print('\n[done] 同一条流水线处理了 2 只票,每只一张分析图 + 一行汇总,换票只改 CONFIG 即可')
```

回测结果

// · matplotlib 真实输出



绘图数据(同一流水线 2 票)

格力电器、海尔智家
2022-2024 各 726 个交易日;同一条
fetch→clean→analyze→plot 流水线产出,只改 CONFIG

四块积木职责单一

fetch 只拉数据、clean 只清洗、analyze 只算(净值/均线/回撤)、plot 只画;改一块不影响别的

run_one 串成流水线

一个总函数按顺序串起四块积木 = 处理一只票的完整流水线;主线清楚,出错一眼定位到哪一块

CONFIG 可复用

所有可调参数(股票/起止/均线天数)集中到 CONFIG;对股票列表循环跑同一条线,换票只改一处、不复制粘贴

格力 vs 海尔(同线对比)

格力 区间 +48.3% / 最大回撤 -24.3%;海尔 +4.5% / -32.6%。同行业同区间收益差约 10 倍、海尔回撤还更深

进阶工具各管一摊

click 做命令行、yaml 存配置、loguru 记日志、cron 定时跑;本课先用最朴素的函数 + print 把思想跑通

ANALYSIS · 这张图告诉我们什么

► 01 模块化 = 把一坨代码拆成职责单一的积木

新手最容易把拉数据、清洗、分析、画图全写成一大坨,结果改哪都怕崩、出错从头查到尾。流水线的思路是拆开:每一步做成1块独立积木(一个函数),每块只管自己那一摊,输入什么、输出什么清清楚楚,像工厂传送带一道接一道。函数其实特别朴素,就是一台小机器:喂进股票代码、吐出行情表,你不用管它肚子里怎么运作。拆开之后,哪一步出问题修哪1块、想升级哪一步换哪1块,别的纹丝不动。前期多花一点功夫,后面反复改、扩展、查 bug 时10 倍还给你。

► 02 接口约定:积木靠『说好吃什么吐什么』对接

积木和积木之间靠什么连?秘诀是约定好接口,说好每块吃进去什么、吐出来什么。比如 clean 约定吃进原始毛坯表、吐出干净规整表;只要这个约定不变,clean 内部你怎么改都不影响前后的积木。就像传送带上每道工序之间的半成品规格固定,某道工序内部换套设备,只要交出的半成品规格不变,下一道照样接得上。约定好接口,是模块化能成立的根基,也是 analyze 只算不画、plot 只画不算这种职责分工的意义所在。

► 03 run_one 串联 + CONFIG 可复用:换票只改一处

4 块积木用一个总函数 run_one 按顺序串起来,就是处理一只票的完整流水线,主线一目了然、出错一眼定位到哪块。更妙的是可复用:把所有要改的参数(股票、起止日期、均线天数)集中进 CONFIG,主程序对 CONFIG 里的每只票循环跑同一条线。想多分析几只票、换个区间,根本不碰流水线本身,只改 CONFIG 一处,整条线自动对所有票重跑,本课一口气处理了格力和海尔两只,一句多余代码都没写。这正是『可复用』对比『复制粘贴3 份』的根本区别。

► 04 格力 vs 海尔:流水线让公平比较毫不费力

两只票跑的是同一条流水线、只在 CONFIG 多写一只的代码,结果却天差地别:格力区间 +48.3%、最大回撤 -24.3%;海尔只 +4.5%、回撤反而更深到 -32.6%。同是家电龙头、同样区间,收益差了约 10 倍。这又一次印证选对具体哪一只差别巨大,而流水线的好处,就是让你能这样毫不费力地把一只一只票拉出来、用完全一致的口径公平比较,这是手动复制粘贴永远做不到的整齐。

► 05 自动化质变 + 写给未来的自己

模块化的价值远不止整理代码。流水线搭好后会带来一个质变:自动化。手动一步步点的时候你被研究拴住,每次都得守着跑完;做成一键流水线后,你可以设它每天收盘自动跑、把图整理好,睡醒复盘就在那儿等你。更深一层:写代码本质是写给未来的自己,今天图省事的一坨,3 个月后你自己都不敢改;今天多花10 分钟拆成清楚的积木,3 个月后的你会无比感激。把复杂拆成各司其职的小积木,不只是写代码的功夫,更是把混乱理出秩序的思维方式。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

A 股	sz.000651 / sh.600690	格力、海尔两只家电龙头跑同一条 fetch→clean→analyze→plot 流水线,各出一张分析图 + 一行汇总
可复用		想换成别的票或别的区间,只改 CONFIG 一处,整条流水线自动重跑,不复制粘贴
每日更新		把流水线接上定时器,每天收盘后自动拉当天数据、出图,人不用守在电脑前
团队协作		模块化后每块职责单一,换数据源只改 fetch、换画法只改 plot,多人协作不打架

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

所有代码焊成一大坨,改一处崩一片

把拉数据、清洗、分析、画图全堆在一个长脚本里,变量互相缠绕,改一个地方常常莫名其妙弄坏另一处。拆成职责单一的函数,每块只管自己那摊,改动被隔离,才不会牵一发动全身。

△ 02

一个函数既算又画,职责不清

让 `analyze` 顺手把图也画了,看着省事,其实把『算』和『画』焊死了:以后想换种画法得动分析逻辑,容易出错。坚持 `analyze` 只算不画、`plot` 只画不算,职责单一,各自独立。

△ 03

参数散落在代码各处,换个值要全文找

股票代码、起止日期、均线窗口如果散写在函数内部,换一次要翻遍全文。集中到一个 `CONFIG` 配置里,所有可调参数一目了然,改一处即可,这也是后面用 `yaml` 配置文件的雏形。

△ 04

复制粘贴处理多只票,而不是循环

想分析三只票就把整段代码复制3份改代码,既臃肿又容易漏改。正确做法是把单只票的流程写成一个函数,再对一个股票列表循环调用,这才是『可复用』的意义。

△ 05

出错了没有任何提示,黑箱跑半天

流水线跑起来如果一声不吭,中途卡在哪一步你完全不知道。在每道工序打一句日志(这节用 `print`,进阶用 `loguru`),让程序边跑边报告进度,出问题能立刻定位到是哪1块。

搭一条好流水线的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 一道工序一个函数,职责单一:fetch 只拿数据、clean 只洗、analyze 只算、plot 只画
- 02 所有可调参数集中到一个 CONFIG(进阶用 yaml 文件),换票换区间只改一处
- 03 处理多只票用循环调用单只票的流水线函数,绝不复制粘贴整段代码
- 04 每道工序打一句日志(print 或 loguru),让程序边跑边报告进度,便于定位问题
- 05 先用最朴素的函数 + print 把思想跑通,再按需引入 click/yaml/loguru/cron 等专门工具

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

01 流水线 = 把一次完整分析拆成 fetch→clean→analyze→plot 四道工序,像传送带一道接一道。

02 模块化 = 每道工序做成1 块职责单一的积木函数,输入输出清楚,改1 块不影响别的。

03 analyze 只算不画、plot 只画不算,职责分清,换算法换画法互不打扰。

04 用一个总函数 run_one 把4 块串成处理单只票的完整流水线。

05 可复用 = 参数集中到 CONFIG,对股票列表循环跑同一条线,换票只改一处、不复制粘贴。

06 进阶工具各管一摊:click 做命令行、yaml 存配置、loguru 记日志、cron 定时跑,先懂思想再上工具。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 为什么要把『拉数据』和『清洗』拆成两个函数,而不是写在一起?各举一个『以后好改』的例子。

Q02 analyze 和 plot 的分工是什么?如果让 analyze 顺手把图也画了,以后会带来什么麻烦?

Q03 『可复用』在这节体现在哪里?想把分析的股票从 2 只加到 5 只,你需要改哪里、不需要改哪里?

Q04 click、yaml、loguru、cron 这四个进阶工具分别解决什么问题?用大白话各说一句。

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Martin 《Clean Code》(2008/Prentice Hall)– 讲『函数职责单一、命名清晰』的经典,模块化思想的源头。
- ▶ Reitz & Schlusser 《The Hitchhiker's Guide to Python》(2016/O'Reilly)– 讲 Python 项目怎么组织模块和结构,实用。
- ▶ click 官方文档(click.palletsprojects.com)– 几分钟学会把脚本变成带参数的命令行工具。
- ▶ loguru 官方文档(github.com/Delgan/loguru)– 一行 import 就能用的现代日志库,比 print 优雅。
- ▶ Percival 《Test-Driven Development with Python》(2017/O'Reilly)– 进阶:怎么给流水线的每块积木写测试,保证重构不出错。

NEXT LECTURE · 下一节预告

DAY 047 TA-Lib 技术指标

数据能拉能存能成流水线了,下一节我们给分析加点专业武器:用行业标准的技术指标库,一键算出均线、相对强弱、快慢线这些经典指标,不用自己推公式,几行代码批量产出几十个指标,给选股和策略当原料。